

NexaFreight Control Tower

Chronological Task Plan · Role Assignments · 15 Weeks · August 2026

● Data Engineer

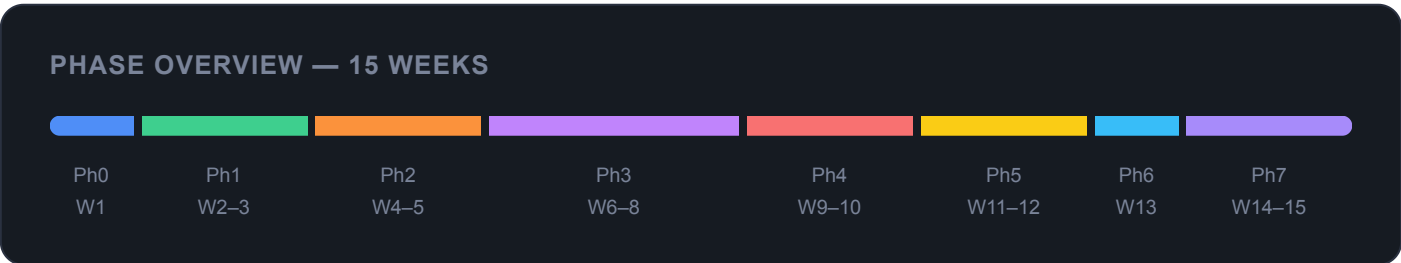
● Backend Engineer

● ML Engineer

● Frontend Engineer

● Full Stack

● DevOps / Infra



PHASE 0

Foundation — Skeleton, Auth, CI

Week 1 · 13 tasks

Duration Week 1

Goal Working skeleton. Zero data, zero features. Everything wires together.

Blocker for All subsequent phases

DAY 1–2 PROJECT SCAFFOLDING

T-001

DevOps / Infra

Create repository + branch strategy

Init GitHub repo. Set default branch to main. Branch naming convention: feat/, fix/, chore/. Add .gitignore (Python + Node + secrets). Add README.md stub.

.gitignore

README.md

T-002

DevOps / Infra

pyproject.toml — all pinned dependencies

Define all backend dependencies with pinned versions. Configure Ruff (linter), mypy (type checker), pytest settings. Add [project.optional-dependencies] dev = [...] group.

pyproject.toml

after T-001

T-003

DevOps / Infra

Pre-commit hooks + GitHub Actions CI

after T-002

Configure `.pre-commit-config.yaml` with: `ruff` (lint + format), `mypy` (type check), `gitleaks` (secret detection). Create `.github/workflows/ci.yml`: on push → install → lint → test. Free for public repos.

`.pre-commit-config.yaml`

`.github/workflows/ci.yml`

T-004

Backend Engineer

Full package structure — all folders + `__init__.py`

after T-002

Create the complete folder tree: `src/nexafreight/` with `models/`, `schemas/`, `adapters/`, `services/`, `ml/`, `routers/`, `workers/`. Every folder gets an `__init__.py`. Create `tests/unit/` and `tests/integration/`.

`src/nexafreight/__init__.py`

`tests/conftest.py`

T-005

Backend Engineer

`config.py` — `pydantic-settings`, all env vars typed

after T-004

Implement `Settings` class using `pydantic-settings BaseSettings`. Every env var typed and documented. `@lru_cache` singleton. Create `.env.example` with instructions for every variable. No hardcoded values anywhere.

`src/nexafreight/config.py`

`.env.example`

T-006

Backend Engineer

`database.py` — `async SQLAlchemy 2.0` + `aiosqlite`

after T-005

Async engine with WAL mode enabled via `connect_args`. `AsyncSessionFactory` with `async_sessionmaker`. `Base = DeclarativeBase()`. FastAPI `get_db()` dependency with `commit/rollback` logic. SQLite path from config.

`src/nexafreight/database.py`

DAY 3-4 ORM MODELS + MIGRATIONS

T-007

Data Engineer

All SQLAlchemy ORM models — complete schema

after T-006

Write all 14 model files: `base.py` (`TimestampMixin`, `ProvenanceMixin`), `user.py`, `location.py`, `order.py`, `order_item.py`, `shipment.py`, `leg.py`, `vessel.py`, `position_report.py`, `event.py`, `disruption.py`, `alert.py`, `reroute_option.py`, `decision.py`, `port_daily_stat.py`, `audit_log.py`. Every column, every index, every relationship defined once here.

`src/nexafreight/models/*.py`

T-008

DevOps / Infra

Alembic init + migration `001_initial_schema`

after T-007

Init Alembic for async SQLite. Configure `alembic/env.py` to use async engine and import all models. Run `alembic revision --autogenerate` to produce

001_initial_schema.py. Verify alembic upgrade head creates all tables. Add make migrate to Makefile.

alembic/env.py

alembic/versions/001_initial_schema.py

DAY 4-5 API SKELETON + AUTH

T-009

Backend Engineer

main.py — FastAPI app factory, lifespan, CORS, router wiring

after T-008

App factory pattern with `@asynccontextmanager` lifespan. On startup: run alembic upgrade, create tables, start APScheduler. CORS middleware configured for `localhost:5173`. Include all routers as stubs (return `200 {"status":"ok"}`). Uvicorn config in Makefile.

src/nexafreight/main.py

src/nexafreight/routers/__init__.py

Makefile

T-010

Backend Engineer

JWT auth — login, refresh, me endpoints

after T-009

POST `/api/auth/login` (form data → JWT). POST `/api/auth/refresh`. GET `/api/auth/me`. Password hashing with `passlib/bcrypt`. Token signing with `python-jose HS256`. `get_current_user` dependency. Role enum: OPERATOR, MANAGER, DIRECTOR. Seed one test user in migration.

src/nexafreight/routers/auth.py

src/nexafreight/schemas/auth.py

T-011

Backend Engineer

Adapter protocols + MockFeedAdapter

after T-004

Write `adapters/protocols.py`: FeedAdapter Protocol, PositionReport dataclass, FeedHealth dataclass, AssetType enum, Provenance enum. Write `adapters/feed/mock.py`: returns deterministic positions for 3 asset types. Used by ALL tests — never touches a real API.

src/nexafreight/adapters/protocols.py

src/nexafreight/adapters/feed/mock.py

T-012

Backend Engineer

tests/conftest.py + 5 financial engine unit tests

after T-011

`conftest.py`: in-memory SQLite test DB, MockFeedAdapter injection, test user fixture. Write `tests/unit/test_financial_engine.py` with 5 tests covering: on-time order (no penalty), 1-day late penalty calculation, demurrage calculation, split-decision trigger, carbon cost. All must PASS before any other code ships.

tests/conftest.py

tests/unit/test_financial_engine.py

T-013

Frontend Engineer

Frontend scaffold — Vite + React + TypeScript + Tailwind + Login page

after T-010

Init Vite project with react-ts template. Install all frontend packages (maplibre-gl, @vis.gl/react-maplibre, @tanstack/react-query, zustand, recharts, tailwindcss, date-fns). Build Login page: email + password form → POST /api/auth/login → store JWT in Zustand (memory only, not localStorage). Protected route wrapper. Basic shell layout (sidebar nav + main).

frontend/src/App.tsx

frontend/src/pages/Login.tsx

frontend/src/store/useAppStore.ts

✓ PHASE 0 — DEFINITION OF DONE

make dev starts on :8000 · npm run dev starts on :5173 · Login works end-to-end (POST → JWT → protected endpoint) · make test passes, zero failures · alembic upgrade head runs clean · gitleaks finds zero secrets in repo

PHASE 1

Data Foundation — Ingest, Consolidate, Pre-compute Routes

Week 2–3 · 11 tasks

Duration **Week 2–3**

Goal **All data ingested. Every shipment has legs. Every leg has a cached GeoJSON route.**

Blocker for **Phase 2 (map needs routes), Phase 3 (ML needs features)**

WEEK 2, DAY 1–2 REFERENCE DATA

T-014

Data Engineer

scripts/02 — Ingest UN/LOCODE into locations table

after T-008

Download UNECE UN/LOCODE CSV. Parse 5-char location codes. Convert lat/lon from DDMM[NS]DDMM[EW] format to decimal degrees. Filter for port, airport, ICD, fixed transport functions. Bulk insert ~100k rows. provenance=CALIBRATED, source=UNECE_UNLOCODE. This runs FIRST — every other script depends on it.

scripts/02_ingest_unlocode.py

data/raw/unlocode/unlocode.csv

T-015

Data Engineer

scripts/03 — Ingest port activity + performance data

after T-014

Parse port_activity.csv (daily vessel counts per port) and maritime_port_performance.csv (dwell times, turnaround). Fuzzy-match port names to UN/LOCODE codes. Compute 90-day rolling baseline for each port. Compute daily congestion_index = today_count / rolling_avg. Populate ports table + port_daily_stats table.

scripts/03_ingest_port_data.py

WEEK 2, DAY 2–4 ORDER INGESTION

T-016

Data Engineer

scripts/01 — Ingest DataCo CSV into orders + order_items

after T-014

Read DataCoSupplyChainDataset.csv with pandas. Snake-case all columns. Map shipping_mode: Standard Class→SEA, Second Class→RAIL, First Class→AIR, Same Day→AIR (express). Compute sla_deadline = order_date + days_for_shipment_scheduled. Map product_category → cargo_class. Derive sla_penalty_pct by customer_segment (Corporate=8%, Consumer=3%, Home Office=5%). Bulk insert orders + order_items. provenance=DERIVED.

scripts/01_ingest_dataco.py

data/raw/dataco/DataCoSupplyChainDataset.csv

T-017

Data Engineer

scripts/04 — Consolidate orders into shipments

after T-016

Group orders by (origin_country, dest_country, mode, cargo_class, week). Fill containers: max 20 orders per shipment or weight limit. For each group: create Shipment record with strictest_sla_deadline = min(order.sla_deadline), total_value_usd, total_weight_kg, reference_number = "NF-{year}-{seq:06d}". Update orders.shipment_id FK. Expected output: 5,000–10,000 shipments from 180k orders.

scripts/04_consolidate_shipments.py

src/nexafreight/services/consolidation.py

WEEK 2 DAY 5 — WEEK 3 DAY 2 ROUTE PRE-COMPUTATION

T-018

Data Engineer

scripts/05 — Plan routes: build all legs with cached GeoJSON geometry

after T-017

For each shipment, build leg sequence by mode. SEA: [FIRST_MILE_ROAD, ORIGIN_HANDLING, SEA_MAIN, DEST_HANDLING, LAST_MILE_ROAD]. AIR: [FIRST_MILE_ROAD, AIR_MAIN, LAST_MILE_ROAD]. ROAD: [ROAD_MAIN]. RAIL: [FIRST_MILE_ROAD, RAIL_MAIN, LAST_MILE_ROAD]. For SEA legs: call searoute-py (pinned 1.4.3), store GeoJSON + distance_nm. Failure → great-circle fallback, route_quality=APPROXIMATE. For ROAD legs: call OpenRouteService (driving-hgv), batch 40/min, cache GeoJSON + distance_km + duration_s. For AIR legs: compute geodesic arc with geopy, lookup duration from OpenFlights. Chain planned_departure/arrival. Compute planned_co2_kg per GLEC factors. This is the most time-consuming script — expect 3-4 hours to run for all lanes.

scripts/05_plan_routes.py

src/nexafreight/adapters/routing/sea_route.py

src/nexafreight/adapters/routing/road_route.py

src/nexafreight/adapters/routing/air_route.py

src/nexafreight/services/route_planner.py

T-019

Data Engineer

scripts/06 — Assign real vessel MMSIs to sea legs

after T-018

For each unique (origin_port, dest_port) sea lane: manually research 2-3 real vessels on that trade route (Maersk/COSCO/Evergreen/Hapag-Lloyd). Look up MMSI on MarineTraffic free vessel search. Insert into vessels table. Assign MMSIs round-robin to sea legs on the same lane. These exact MMSIs = the AISStream subscription list. Goal: 20-50 MMSIs total covering all active sea lanes.

scripts/06_assign_vessels.py

T-020

Data Engineer

scripts/07 — Assign flight numbers + durations to air legs

after T-018

For each unique (origin_airport, dest_airport) air lane: pick a real cargo carrier (Cathay Cargo, FedEx, DHL, Korean Air Cargo). Get typical flight duration from OpenFlights database (free CSV). Update legs.flight_number and legs.planned_duration_hours. ReplayFlightAdapter will use planned_duration_hours for geodesic interpolation at runtime.

scripts/07_assign_flights.py

data/raw/openflights/routes.dat

WEEK 3, DAY 3–5 SHIPMENT API ENDPOINTS

T-021

Backend Engineer

GET /api/shipments — paginated list with filters

after T-017

Implement shipments list endpoint. Filters: status, mode, alert=true. Pagination: page + size. Response: ShipmentListItem (id, reference, origin, dest, mode, status, strictest_sla_deadline, revised_eta, provenance). Write ShipmentListItem Pydantic schema. Add DB indexes on (status, mode).

src/nexafreight/routers/shipments.py

src/nexafreight/schemas/shipment.py

T-022

Backend Engineer

GET /api/shipments/{id} + /route + /events endpoints

after T-021

ShipmentDetail: full shipment + all legs (ordered by sequence) + all orders (summary) + recent events. GET /route: returns all leg geometries as GeoJSON FeatureCollection, tagged by mode + route_quality. GET /events: shipment event log, descending. All responses include provenance on every sub-object.

src/nexafreight/routers/shipments.py

T-023

Backend Engineer

Unit tests — consolidation + route planner

after T-022

test_consolidation.py: verify correct grouping by mode/cargo_class/week, strictest_sla computed correctly, weight limit enforced. test_route_planner.py: verify leg sequence for each mode, correct leg_type assignment, planned_departure chain, route_quality flag on fallback. All tests use in-memory SQLite + fixtures — no external APIs called.

tests/unit/test_consolidation.py

tests/unit/test_route_planner.py

T-024

Full Stack

Frontend — ShipmentList page (API-connected, no map yet)

after T-022

Build ShipmentList.tsx: table of active shipments, fetched via TanStack Query (auto-refetch every 30s). Columns: reference, origin→dest, mode icon, status badge, SLA deadline, ETA. Filter

controls (mode, status). Pagination. Click row → navigate to ShipmentDetail (stub for now). Verify API data renders correctly.

`frontend/src/pages/Dashboard.tsx`

`frontend/src/components/shipments/ShipmentList.tsx`

`frontend/src/api/shipments.ts`

✓ PHASE 1 — DEFINITION OF DONE

GET /api/shipments/42/route returns valid GeoJSON FeatureCollection · Sea routes visually pass through correct straits and canals (not through land) · Road routes follow actual roads · Every leg has provenance label · 5,000+ shipments in DB derived from DataCo orders

PHASE 2 Live Map — The WOW Moment

Week 4–5 · 10 tasks

Duration **Week 4–5**

Goal **Real ships move on a real map. Trucks inch along roads. Flights arc across continents.**

Blocker for **Phase 4 (alerts need live positions)**

WEEK 4, DAY 1–2 FEED ADAPTERS

T-025

Data Engineer

scripts/08 — Download historical AIS data to Parquet

after T-019

Download historical AIS from NOAA Marine Cadastre (US waters, free) OR Danish Maritime Authority (global, free research license). Filter to only your vessel MMSIs (from vessels table). Convert CSV → Parquet using pyarrow (10x smaller). Store in `data/raw/ais_historical/`. This dataset powers `ReplayFeedAdapter` when `USE_LIVE_AIS=false`.

`scripts/08_download_ais_history.py`

`data/raw/ais_historical/*.parquet`

T-026

Backend Engineer

adapters/feed/aisstream.py — AISStream WebSocket adapter

after T-025

Implement `AISStreamAdapter`: async WebSocket connection to `wss://stream.aisstream.io/v0/stream`. Subscribe with API key + `FiltersShipMMSI` (your MMSI list) + `FilterMessageTypes=["PositionReport"]`. On each message: parse JSON, extract `lat/lon/speed/heading`, upsert in-memory cache `dict[MMSI → PositionReport]`. On disconnect: exponential backoff (1s→2s→4s→...max 60s), resubscribe within 3s. Track `FeedHealth`. Enable `permessage-deflate` compression to avoid Sept 2026 bandwidth limit.

`src/nexafreight/adapters/feed/aisstream.py`

T-027

Backend Engineer

adapters/feed/replay_ais.py — Parquet replay adapter

after T-026

`ReplayFeedAdapter` reads historical AIS Parquet files. Advances a virtual clock at configurable speed (e.g. 60x real time for demos). Serves position as if live for each MMSI.

Provenance=REPLAYED. Used when USE_LIVE_AIS=false in .env. Identical interface to AISStreamAdapter — no business logic changes needed to switch between them.

```
src/nexafreight/adapters/feed/replay_ais.py
```

T-028

Backend Engineer

adapters/feed/truck_sim.py + replay_flight.py — simulation adapters

after T-027

TruckSimAdapter: given a road leg (cached ORS GeoJSON polyline + departure time + planned duration), compute interpolated position using Shapely

LineString.interpolate(progress, normalized=True). Compute heading from current→next point. Provenance=SIMULATED. ReplayFlightAdapter: given air leg (geodesic arc GeoJSON + departure time + planned duration), interpolate position along arc using geopy. Both write to position_reports every 30s via position_interpolator worker.

```
src/nexafreight/adapters/feed/truck_sim.py
```

```
src/nexafreight/adapters/feed/replay_flight.py
```

WEEK 4, DAY 3–5 BACKGROUND WORKERS + SSE

T-029

Backend Engineer

workers/ais_listener.py — persistent asyncio background task

after T-026

Persistent asyncio Task started in lifespan. Runs AISStreamAdapter._run_with_backoff().

On each position update: write to position_reports table (upsert by MMSI + timestamp).

Update in-memory position cache. Update feed health. Expose get_position_tracker() singleton for other workers and routers to query the cache.

```
src/nexafreight/workers/ais_listener.py
```

T-030

Backend Engineer

workers/position_interpolator.py — 30s interval truck + flight

after T-028, T-029

APScheduler job: runs every 30 seconds. Queries all IN_PROGRESS ROAD and AIR legs. For each: call TruckSimAdapter or ReplayFlightAdapter. Write PositionReport (provenance=SIMULATED) to position_reports table. Update shipment.current_position cache. Cleans up STALE entries older than 24h from position_reports.

```
src/nexafreight/workers/position_interpolator.py
```

T-031

Backend Engineer

GET /api/map/positions/stream — SSE endpoint

after T-029, T-030

Server-Sent Events endpoint. Frontend connects once via `EventSource('/api/map/positions/stream')`. Every 5 seconds: gather all current positions from in-memory cache → serialize → yield as `"data: {json}\n\n"`. Each position includes: `asset_type`, `asset_id`, `lat`, `lon`, `speed_knots`, `heading_deg`, `provenance`, `source`. Also implement GET `/api/map/positions/snapshot` (one-time load), GET `/api/map/routes` (all active route GeoJSONs), GET `/api/map/ports`, GET `/api/map/feed-health`.

```
src/nexafreight/routers/map.py
```

WEEK 5 MAP FRONTEND

T-032

Frontend Engineer

FreightMap.tsx — MapLibre GL JS v6 with MapTiler tiles

after T-031

Implement the map component using `@vis.gl/react-maplibre v8.1.1`. MapTiler dark dataviz tile style (free tier, 100k loads/month). Initial view: `lon=0`, `lat=20`, `zoom=2`. Add route Source + Layer for each mode: `sea=blue dashed`, `air=orange solid`, `road=green solid`. Fetch routes from GET `/api/map/routes` on load. Draw all active leg geometries. Route data from cached GeoJSON in DB — instant load, no computation.

```
frontend/src/components/map/FreightMap.tsx
```

```
frontend/src/components/map/RouteLayer.tsx
```

T-033

Frontend Engineer

Live asset markers — SSE hook + animated icons with heading + provenance badge

after T-032

Implement `useSSEPositions()` hook: creates `EventSource`, parses `POSITION_UPDATE` messages, sets state. Each position renders as a Marker with: SVG icon by `asset_type` (ship/plane/truck), CSS rotation = `heading_deg`, click → show shipment side panel. Provenance badge: `REAL=no badge`, `SIMULATED=yellow "S" pill`, `REPLAYED=grey "R" pill`. Markers update every 5s as SSE pushes new data. Smooth position transition via CSS transition.

```
frontend/src/api/sse.ts
```

```
frontend/src/components/map/ShipLayer.tsx
```

```
frontend/src/components/map/FlightLayer.tsx
```

```
frontend/src/components/map/TruckLayer.tsx
```

T-034

Full Stack

Integration test — SSE delivers positions on schedule

after T-033

Integration test: start test server, connect SSE client, verify POSITION_UPDATE received within 6 seconds. Verify ship positions match AIS cache. Verify truck positions change on consecutive messages. Verify provenance field present on every position object. Verify feed-health endpoint returns GREEN for all adapters. Run with MockFeedAdapter so no live API needed in CI.

tests/integration/test_shipment_api.py

✓ PHASE 2 — DEFINITION OF DONE

Open dashboard — ships move in real-time (REAL or REPLAYED) · Trucks inch along road polylines every 30 seconds · Flights move along curved geodesic arcs · No asset marker ever crosses land on a sea route · Feed health panel shows per-adapter status · Click any asset → side panel shows shipment summary

PHASE 3**ML Models — Delay, ETA, Demand**

Week 6–8 · 9 tasks

Duration **Week 6–8**Goal **Three trained models. SHAP explanations. All predictions served via API. All beat baseline.**Blocker for **Phase 4 (alerts use ML predictions)****WEEK 6 FEATURE ENGINEERING + DELAY CLASSIFIER**

T-035

ML Engineer

Feature engineering — build training dataset from DB

after T-015, T-017

Query SQLite to join: orders + legs + port_daily_stats. Build feature matrix: shipping_mode_encoded, origin/dest country codes, planned_transit_days, cargo_value_usd binned, cargo_class_encoded, origin/dest port congestion_index (on order_date), departure_month, departure_weekday, distance_km, sla_buffer_days. Time-based split: train before 2017-06-01, val 2017-06-01 to 2017-12-31, test 2018-01+. Save feature_schema.json. Compute baseline AUC (historical late rate per lane).

scripts/09_train_delay_classifier.py

T-036

ML Engineer

Train LightGBM delay classifier + Optuna tuning + SHAP

after T-035

Train LightGBM classifier on time-split training set. Optuna: 50 trials maximizing val ROC-AUC. Fit SHAP TreeExplainer on best model. Evaluate on holdout test set: must beat baseline_roc_auc. Save model.joblib, feature_schema.json, metadata.json (git_sha, data_hash, val_roc_auc, test_roc_auc, baseline_roc_auc, trained_at, n_train_samples). Fail loudly if test_roc_auc ≤ baseline_roc_auc.

models/delay_classifier/model.joblib

models/delay_classifier/metadata.json

WEEK 7 ETA QUANTILE MODEL

T-037

ML Engineer

Train LightGBM quantile ETA model (P10 / P50 / P85)

after T-036

Target = actual_transit_days - planned_transit_days (the residual). Three separate LightGBM models: alpha=0.10, 0.50, 0.85. Same features as classifier PLUS: current_leg_progress, days_elapsed_so_far, current_dest_congestion_index. Evaluate on pinball loss per quantile. Coverage check: P10-P85 interval must contain actual ~75% of the time on test set. Save three model files + shared metadata. P85 is the conservative estimate used for SLA risk checks.

scripts/10_train_eta_model.py

models/eta_quantile/model.joblib

models/eta_quantile/metadata.json

T-038

ML Engineer

Train StatsForecast AutoETS demand model

after T-036

Aggregate weekly order volume per (product_category × origin_region × dest_region). Fit StatsForecast AutoETS (automatic exponential smoothing with trend + seasonality). Generate 30/60/90-day forecasts with confidence intervals. Evaluate: holdout MAPE per category × lane. Save model and metadata. No PyStan, no cmdstan — pure Python, ~30 seconds to fit all series.

scripts/11_train_demand_forecast.py

models/demand_forecast/model.joblib

WEEK 7-8 ML INFERENCE LAYER + API

T-039

ML Engineer

ml/registry.py + inference wrappers for all 3 models

after T-036, T-037, T-038

registry.py: loads all models from /models/ dir at app startup. Validates metadata.json (git_sha, data_hash, metrics). Exposes get_delay_classifier(), get_eta_model(), get_demand_forecast(). ml/delay_classifier.py: _extract_features(shipment) → feature vector → predict_proba → SHAP values → top 5 factors. ml/eta_model.py: three model calls → P10/P50/P85 ETAs → sla_risk label. ml/demand_forecast.py: forecast per category × lane → chart-ready JSON. Raw feature vectors NEVER appear in API responses.

src/nexafreight/ml/registry.py

src/nexafreight/ml/delay_classifier.py

src/nexafreight/ml/eta_model.py

src/nexafreight/ml/demand_forecast.py

T-040

Backend Engineer

GET /api/shipments/{id}/predict endpoint

after T-039

Call delay_classifier.predict(shipment) + eta_model.predict(shipment). Return: delay_probability, is_high_risk, top_factors (SHAP with direction + plain-English label), p10_eta, p50_eta, p85_eta, sla_risk (ON_TIME/LOW/MEDIUM/HIGH/BREACH),

model_version, provenance=DERIVED. Add DB index on (shipment_id) for fast feature extraction. Serve 503 with clear message if models not loaded.

```
src/nexafreight/routers/shipments.py
```

T-041

Frontend Engineer

Frontend — Risk badges + ETA ranges + SHAP factors on shipment cards

after T-040

Update ShipmentList cards: add risk badge (LOW=green, MEDIUM=yellow, HIGH=red) from delay_probability. Add ETA range: "Dec 15–21" (P10→P85 dates). ShipmentDetail page: SHAP factors panel — horizontal bar chart showing top 5 feature contributions with direction labels. Analytics page: DemandForecast.tsx shows 30/60/90-day forecast lines per category via Recharts LineChart.

```
frontend/src/components/shipments/ShipmentDetail.tsx
```

```
frontend/src/components/analytics/DemandForecast.tsx
```

T-042

ML Engineer

Unit tests — delay classifier + ETA model

after T-039

test_delay_classifier.py: test feature extraction with known shipment fixture, test prediction output schema, test SHAP factors present + sum to ~1, test provenance=DERIVED on output. test_eta_model.py: test $P_{10} \leq P_{50} \leq P_{85}$ always, test sla_risk label logic. All tests use saved model artifacts (not retrained during test). Run in CI in <10 seconds.

```
tests/unit/test_delay_classifier.py
```

```
tests/unit/test_eta_model.py
```

T-043

Full Stack

GET /api/analytics/demand endpoint + DuckDB integration

after T-038, T-039

First DuckDB query: connect DuckDB to the SQLite file directly (duckdb.connect reads SQLite natively). Query aggregated weekly order volumes. Join with demand_forecast model output. Return chart-ready JSON: [{category, lane, week, actual_volume, forecast_p50, forecast_lower, forecast_upper}]. Verify DuckDB columnar query is faster than SQLAlchemy ORM for aggregations.

```
src/nexafreight/routers/analytics.py
```

✅ PHASE 3 — DEFINITION OF DONE

All 3 models beat their naive baselines on holdout test set · SHAP explanations make intuitive sense (congested port → increased risk) · Every prediction includes model_version + provenance=DERIVED · Risk badges visible on all active shipment cards · ETA range displayed as date span (not a single date) · Demand forecast chart renders correct category data

PHASE 4

Alert + Financial Engine — Disruption → Alert → Impact

Week 9–10 · 9 tasks

Duration **Week 9–10**

Goal **Disruptions auto-detected. Alerts fired. Per-order financial impact computed. Operator sees \$ exposure.**

Blocker for **Phase 5 (rerouting needs alerts as input)**

WEEK 9, DAY 1-3 DETECTION SERVICES

T-044

Backend Engineer

services/disruption_detector.py — AIS deviation detection

after T-029

Called on every AIS position update. Computes planned_progress (elapsed_time / planned_duration) and actual_progress (distance_sailed_along_route / total_route_distance). If planned_progress - actual_progress > VESSEL_DELAY_THRESHOLD_PCT (default 0.10): create DISRUPTION record with type=VESSEL_DELAY, estimated_delay_hours, confidence=0.75, source=AIS_DEVIATION. Idempotent: do not duplicate disruptions for the same vessel + same leg.

src/nexafreight/services/disruption_detector.py

T-045

Backend Engineer

workers/port_monitor.py — daily congestion check → disruptions

after T-015

APScheduler cron: daily at 6am. For each port in ports table: fetch today's port_daily_stats.vessel_count. Compute congestion_index = today_count / rolling_90_day_avg. If > PORT_CONGESTION_THRESHOLD (1.4): find all PLANNED or IN_TRANSIT shipments with dest_port = this port. For each: create DISRUPTION(type=PORT_CONGESTION, source=PORT_DATA). Log count of disruptions created.

src/nexafreight/workers/port_monitor.py

T-046

Backend Engineer

services/financial_engine.py — per-order impact calculation

after T-017

calculate_option_financials(shipment, revised_eta, freight_delta, port_delta, demurrage_extra_days, reroute_cost, co2_delta_kg) → OptionFinancials. For each order in shipment: is_breached = revised_eta > order.sla_deadline. days_late = max(0, delta.days). sla_penalty = revenue × penalty_pct × days_late. Total = freight_delta + port_delta + demurrage + reroute_cost + sum(sla_penalties) + co2_cost. Demurrage = max(0, extra_days - free_days) × daily_rate from port leg.

src/nexafreight/services/financial_engine.py

src/nexafreight/services/sla_checker.py

T-047

Backend Engineer

services/alert_engine.py — disruption → alert with financials

after T-044, T-046

On new DISRUPTION: compute revised_eta for affected shipment. Call financial_engine.calculate_option_financials(accept_delay_scenario). Determine severity: CRITICAL if exposure > \$20k or any order > 5 days late, HIGH if > \$5k or any breach,

MEDIUM if > \$1k or ETA within 2 days of SLA. Create ALERT record with sla_breaches_json (per-order detail) and financial_exposure_usd. Idempotent: one alert per (shipment, disruption) pair.

src/nexafreight/services/alert_engine.py

T-048

Backend Engineer

workers/sla_monitor.py — 15-min SLA breach scan

after T-047

APScheduler interval: every 15 minutes. For every IN_TRANSIT shipment: get latest revised_eta (from AIS cache or interpolation). Call sla_checker.check_all_orders(shipment, revised_eta). If any new breach detected (not already alerted): create ALERT(type=SLA_RISK). If existing alert's disruption is now resolved (eta improved): update alert.status=RESOLVED. Log summary counts.

src/nexafreight/workers/sla_monitor.py

WEEK 9 DAY 4 — WEEK 10 DAY 2 ALERT API + MANUAL DISRUPTIONS

T-049

Backend Engineer

Alert + Disruption API endpoints

after T-047, T-048

POST /api/disruptions: manual disruption entry (OPERATOR role). Validates locode exists, creates DISRUPTION, immediately triggers alert_engine. GET /api/disruptions: list with filters. GET /api/alerts: sorted by severity DESC, created_at DESC. Filters: status, severity (comma-separated). GET /api/alerts/{id}: full detail including per-order SLA breakdown from sla_breaches_json. PATCH /api/alerts/{id}/acknowledge: updates status, logs acknowledger + timestamp.

src/nexafreight/routers/alerts.py

src/nexafreight/routers/disruptions.py

src/nexafreight/schemas/alert.py

src/nexafreight/schemas/disruption.py

T-050

Frontend Engineer

Frontend — AlertPanel.tsx with per-order SLA breakdown

after T-049

AlertPanel.tsx: right-side drawer on dashboard. Alert cards sorted by severity (CRITICAL=red, HIGH=orange, MEDIUM=yellow). Each card: disruption type icon, shipment reference, ETA impact (+N days), financial exposure (\$), severity badge, time since triggered. Click card → expand to full detail: per-order table (order ID, SLA deadline, revised ETA, days late, penalty \$, breach/safe icon). Acknowledge button. Manual disruption form button.

frontend/src/components/alerts/AlertPanel.tsx

frontend/src/components/alerts/AlertCard.tsx

frontend/src/pages/AlertCenter.tsx

frontend/src/api/alerts.ts

T-051

Backend Engineer

Unit tests — financial engine + SLA checker

after T-046

test_financial_engine.py (expand from T-012): test demurrage free-day logic, test SLA penalty at 1/2/5 days late, test carbon cost calculation, test multi-order shipment total, test split-decision net savings. test_sla_checker.py: test breach detection with exact deadline equality, test all-safe scenario, test all-breach scenario. All with known numeric values — no approximations.

tests/unit/test_financial_engine.py

tests/unit/test_sla_checker.py

T-052

Full Stack

Integration test — full disruption → alert lifecycle

after T-049, T-051

test_alert_lifecycle.py: POST manual PORT_STRIKE disruption → verify DISRUPTION created → verify ALERT created within 1s → verify alert.financial_exposure_usd > 0 → verify sla_breaches_json has correct per-order detail → PATCH acknowledge → verify status=ACKNOWLEDGED → verify audit_log entry created. Run in CI with MockFeedAdapter and test DB.

tests/integration/test_alert_lifecycle.py

✓ PHASE 4 — DEFINITION OF DONE

POST manual disruption → ALERT appears within 30 seconds · Alert shows correct financial exposure (demurrage + SLA penalties) · Per-order table correctly identifies which orders breach and which are safe · Financial engine unit tests pass with known numeric values · Integration test passes in CI

PHASE 5

Rerouting Engine — 3 Options, Approve, Execute, Audit

Week 11–12 · 8 tasks

Duration **Week 11–12**

Goal **Operator sees 3 options with full financials. Approves one. Legs update. Audit log written. Map shows new route.**

Blocker for **Phase 7 (analytics need decision data)**

WEEK 11, DAY 1–2 CORRIDOR DATA + REROUTE ENGINE

T-053

Data Engineer

Seed corridor_alternatives table — pre-built rerouting data

after T-018, T-019

For each major DataCo sea/air lane: define 2-3 pre-built alternatives as DB rows. Each alternative has: option_key, name, when_to_use (JSON list of disruption types), new_legs_template (JSON), cost_delta_factor, time_delta_hours, co2_delta_factor. Examples: Shanghai→Long Beach via Oakland, via Seattle+Rail, via Air. This is DATA not code — non-engineers can update it without deployment. Seed via Alembic data migration.

alembic/versions/002_seed_corridors.py

T-054

Backend Engineer

services/reroute_engine.py — generate 3 scored options

after T-053, T-046

generate_options(alert) → list[RerouteOption]. Always include ACCEPT_DELAY as Option A (baseline). Query corridor_alternatives filtered by disruption.type in alt.when_to_use. For each qualifying alternative: compute revised_eta, call financial_engine for cost/SLA/CO2. Evaluate SPLIT if some orders breach and others don't (net_savings = penalties - air_cost). Rank by total_financial_impact. Mark is_ai_recommended=True on best option that satisfies SLA. Return top 3. Save all options to reroute_options table.

src/nexafreight/services/reroute_engine.py

T-055

Backend Engineer

services/decision_executor.py — apply approved decision

after T-054

execute_decision(alert_id, option_id, user): fetch reroute_option. If DIVERT/MODAL_SHIFT: cancel remaining PLANNED legs (status=REPLACED, not deleted). Create new legs from option.new_legs_json. Increment shipment.route_version. If SPLIT: create new child shipment for priority orders with air legs. Update parent shipment to standard_orders only. If ACCEPT_DELAY: update revised_eta only. Write IMMUTABLE audit_log entry. Recalculate shipment P&L. Outbox pattern: external notifications queued, not synchronous.

src/nexafreight/services/decision_executor.py

WEEK 11 DAY 3 — WEEK 12 DAY 2 DECISION API + FRONTEND

T-056

Backend Engineer

Decision API endpoints — options + approve + audit history

after T-054, T-055

GET /api/alerts/{id}/options: trigger reroute_engine.generate_options() if not already generated, return 3 RerouteOption objects with full financials. POST /api/alerts/{id}/approve: body={option_id, notes}. Requires OPERATOR role. Calls decision_executor.execute_decision(). Updates alert.status=APPROVED. Returns {status, new_eta, shipment_route_version}. GET /api/decisions: paginated audit history with filters. GET /api/decisions/{id}: full decision detail including all 3 options that were presented.

src/nexafreight/routers/decisions.py

src/nexafreight/schemas/reroute.py

T-057

Frontend Engineer

Frontend — RerouteOptions.tsx — 3 option cards with Approve

after T-056

Full-screen modal opened from high-severity alert card. Top section: disruption details + affected shipment. Middle: per-order SLA breach table. Bottom: 3 option cards side by side. Each card: name, revised ETA, cost delta (green if savings, red if additional), SLA breaches count, CO2 delta, financial impact total. AI recommended option: ★ badge + subtle highlight. [Approve] button on each card → confirmation dialog: "Approve [Option Name]? This cannot be undone." → POST /api/alerts/{id}/approve → success toast → modal closes → map re-fetches routes (shows new path). Disable all approve buttons until a choice is confirmed.

frontend/src/components/alerts/RerouteOptions.tsx

T-058

Frontend Engineer

Frontend — AuditLog.tsx page

after T-056

AuditLog page at /audit. Table: timestamp, action_type, entity, performed_by, details. Filterable by date range, action_type, user. Decision rows are expandable: show all 3 options that were presented, which was chosen, financial impact of chosen option vs rejected options. Export to CSV button. This is the compliance/review screen for managers. Read-only — no actions available.

`frontend/src/pages/AuditLog.tsx`

T-059

Backend Engineer

Unit tests — reroute engine + decision executor

after T-054, T-055

test_reroute_engine.py: test ACCEPT_DELAY always present as option, test corridor filtering by disruption type, test SPLIT recommended when penalties > air_cost, test AI recommendation = lowest financial impact option satisfying SLA.

test_decision_executor.py: test legs updated on DIVERT (old legs REPLACED not deleted), test route_version incremented, test audit_log entry created, test idempotency (second call to execute raises error).

`tests/unit/test_reroute_engine.py`

T-060

Full Stack

Integration test — full lifecycle: disruption → alert → approve → map updates

after T-056, T-059

test_decision_workflow.py: POST disruption → GET alert → GET options (verify 3 returned with financials) → POST approve Option B → GET shipment/route (verify new GeoJSON includes new legs) → GET decisions (verify immutable entry exists) → attempt POST approve again (verify 400 error — already decided) → verify audit_log has entry with all 3 options recorded. The definitive end-to-end test. Must pass before Phase 5 is marked complete.

`tests/integration/test_decision_workflow.py`**✓ PHASE 5 — DEFINITION OF DONE**

Inject disruption → 3 options appear with correct financials · Approve Option B → legs updated → map shows new route → audit log entry created · SPLIT option appears when some orders breach and others do not · Approved decision cannot be re-approved (idempotency) · Full lifecycle integration test passes in CI

PHASE 6**LLM Copilot — Explains, Never Decides**

Week 13 · 4 tasks

Duration **Week 13**Goal **Operator can ask questions about any shipment. LLM answers from structured context only.**Note **Lowest priority. Do not let this consume time from Phases 1–5.****WEEK 13, DAY 1–2 LLM ADAPTERS**

T-061

Backend Engineer

adapters/llm/gemini.py + adapters/llm/ollama.py

after T-039

GeminiAdapter: calls google-generativeai Gemini Flash (1M tokens/day free).
 OllamaAdapter: calls local Ollama HTTP API (llama3.1:8b). Both implement LLMAdapter protocol: generate(system_prompt, user_message) → str. Selected by USE_GEMINI env flag. GeminiAdapter falls back to OllamaAdapter on HTTP 429 rate limit. Both log input_hash + output_hash to audit_log for every call. Neither adapter has any DB write access.

src/nexafreight/adapters/llm/gemini.py

src/nexafreight/adapters/llm/ollama.py

T-062

Backend Engineer

services/copilot.py — context builder + POST /api/copilot/ask

after T-061

build_copilot_context(shipment_id): queries shipment state, active alerts, ML prediction, financial snapshot, applicable SOP rules. Serializes to structured JSON string. System prompt: analyst persona + strict rules (answer only from context, never invent figures, cannot approve actions). POST /api/copilot/ask: validates shipment_id belongs to requesting user's accessible data. Calls LLM adapter. Returns {answer, model_used, context_fields_used}. Serve 503 if both Gemini and Ollama unavailable.

src/nexafreight/services/copilot.py

src/nexafreight/routers/copilot.py

T-063

Frontend Engineer

Frontend — CopilotPanel in ShipmentDetail

after T-062

Collapsible panel at bottom of ShipmentDetail page. Text input: "Ask about this shipment...". Submit → POST /api/copilot/ask → show answer with model_used badge (Gemini Flash / Ollama llama3.1:8b). Show loading spinner during response. Expandable "Data used" section: lists which context fields were included in the prompt. Error state: if 503 show "AI copilot currently unavailable." Never show a loading state for more than 15 seconds — timeout with friendly message.

frontend/src/components/shipments/ShipmentDetail.tsx

T-064

Full Stack

Copilot smoke tests — 3 canned questions with MockLLM

after T-062

Use a MockLLMAdapter that echoes the context as its response. Test 1: "Why is this shipment at risk?" → verify ML prediction data appears in context. Test 2: "What does Option B cost?" → verify financials appear in context. Test 3: question about data not in context → verify copilot gracefully acknowledges it cannot answer. All tests: verify audit_log entry created for each call. Verify copilot endpoint returns 403 if user requests another user's shipment.

tests/unit/test_copilot.py

 **PHASE 6 — DEFINITION OF DONE**

"Why is this shipment at risk?" → correct answer citing ML prediction + port congestion data · "What does Option B cost?" → correct dollar figure from financials · "What is the weather in Rotterdam?" → "I don't have weather data" · Works fully offline with Ollama (USE_GEMINI=false, ollama pull llama3.1:8b)

PHASE 7

Analytics + Polish + Demo Script

Week 14–15 · 8 tasks

Duration **Week 14–15**

Goal **Financial scorecard, ESG report, SLA heatmap, demand charts. 10-minute demo script ready. README complete.**

WEEK 14, DAY 1–3 ANALYTICS API (DUCKDB)

T-065

Data Engineer

DuckDB analytics queries — scorecard, ESG, SLA, P&L

after T-055

Connect DuckDB to the SQLite file directly (zero ETL — DuckDB reads SQLite natively). Write four analytical queries: (1) Financial Scorecard: total_shipments, on_time_pct, total_revenue, total_margin, total_demurrage, total_sla_penalties by date range. (2) ESG Report: total_co2_kg by mode, trend by week, comparison vs air-only baseline. (3) SLA Performance: on_time_pct heatmap by mode × dest_region. (4) P&L Breakdown: revenue vs all cost categories stacked. All queries < 1s on DuckDB columnar engine.

```
src/nexafreight/routers/analytics.py
```

T-066

Frontend Engineer

Frontend — FinancialScorecard.tsx + ESGReport.tsx + SLA heatmap

after T-065

Analytics page: three main panels. FinancialScorecard: KPI cards (total shipments, on-time %, total margin \$, demurrage total \$, SLA penalties \$). Date range filter. ESGReport: stacked bar chart (CO2 by mode per week), total vs air-only baseline comparison metric, CO2 saved vs worst-case. SLA Performance: grid heatmap (mode × dest region, colour intensity = on-time%). All charts via Recharts. All data from GET /api/analytics/* endpoints. Auto-refresh every 5 minutes.

```
frontend/src/components/analytics/FinancialScorecard.tsx
```

```
frontend/src/components/analytics/ESGReport.tsx
```

```
frontend/src/pages/Analytics.tsx
```

T-067

DevOps / Infra

DB indexes — hot query path tuning

after T-065

Profile API response times after all features built. Add missing SQLite indexes on hot paths: (shipments.status, shipments.primary_mode), (legs.shipment_id, legs.route_version, legs.sequence), (position_reports.asset_type, position_reports.asset_id, position_reports.reported_at DESC), (orders.shipment_id), (alerts.status, alerts.severity). Create Alembic migration 003_add_indexes.py. Target: all list endpoints < 200ms on 10k shipments.

WEEK 14 DAY 4 — WEEK 15 DAY 2 FEED HEALTH + POLISH

T-068

Full Stack

Feed health dashboard + GET /api/shipments/{id}/financials endpoint

after T-066

Frontend: Feed Health widget on dashboard (small bottom bar). Shows per-adapter: name, provenance badge, last_success (relative time), messages_received, is_healthy (green/red dot). Turns yellow if last_success > 5 min ago. Backend: GET /api/shipments/{id}/financials returns current P&L snapshot: revenue, freight_cost, port_handling, demurrage_accrued, sla_penalties_accrued, carbon_cost, gross_margin, margin_pct. Displayed in ShipmentDetail sidebar.

src/nexafreight/routers/shipments.py

frontend/src/components/shipments/ShipmentDetail.tsx

T-069

Frontend Engineer

ShipmentDetail page — complete implementation

after T-068

Full ShipmentDetail.tsx: left column: leg timeline (each leg with mode icon, status, planned vs actual times, carrier, vessel/flight/truck info). Right column: ML prediction panel (risk badge + ETA range + SHAP chart) + Financial snapshot + active alerts. Bottom: Order table (all orders in shipment with SLA status per order). CopilotPanel (collapsible). Event log (timeline of all events). Route version history (if rerouted, show "Route v2 applied on [date] — Approved by [user]").

frontend/src/components/shipments/ShipmentDetail.tsx

frontend/src/components/shipments/OrderTable.tsx

WEEK 15, DAY 3–5 DEMO + DOCUMENTATION

T-070

Full Stack

Demo script — scripted 10-minute walkthrough

after T-069

Write a demo_script.md. Pre-stage: create 3 specific shipments with known MMSIs, assign to DataCo orders. Minute 0-2: show live map, call out real ship moving. Minute 2-3: open ShipmentDetail, show ML prediction (HIGH risk, SHAP shows Rotterdam congestion). Minute 3: fire pre-staged PORT_CONGESTION disruption via manual POST. Minute 3-4: alert appears, show financial exposure (\$18,400). Minute 4-6: open rerouting options, walk through each card, note AI recommendation. Minute 6-7: approve Option B. Minute 7-8: map refreshes showing new route polyline. Minute 8-10: show Analytics page (ESG, scorecard, SLA heatmap). Copilot: "Why was this shipment rerouted?" → clear answer.

demo_script.md

scripts/seed_demo_data.py

T-071

DevOps / Infra

README.md — architecture diagram, setup instructions, data sources

after T-070

Complete README: what NexaFreight is (1 paragraph, no jargon). Architecture diagram (ASCII or Mermaid). Prerequisites: Python 3.11, Node 20, API keys (with links to free registration). Step-by-step setup: git clone → pip install -e ".[dev]" → npm install → copy .env.example → get API keys → make migrate → make seed → make train → make dev. Data sources section: DataCo (Kaggle link), UN/LOCODE (UNECE link), NOAA AIS (link), AISStream.io, OpenRouteService, MapTiler. Screenshots of dashboard, alert workflow, analytics.

README .md

T-072

DevOps / Infra

Secret rotation + final security audit

after T-071

Rotate all API keys that were used during development (AISStream, ORS, MapTiler, Gemini). Generate new SECRET_KEY with openssl rand -hex 32. Run gitleaks on entire git history — must return zero findings. Verify .env is in .gitignore and has never been committed. Verify .env.example has no real values. Run bandit security linter: bandit -r src/. Check all CORS origins are explicit (no wildcard *). Final: run make check → lint + test → must be green.

.env.example .gitignore

✓ PHASE 7 — DEFINITION OF DONE

Analytics page: Financial Scorecard + ESG + SLA heatmap + Demand Forecast all render correct data · Demo script runs cleanly in 10 minutes without rehearsal surprises · README: a new developer can set up from scratch following only the README · gitleaks: zero secrets found in git history · make check: lint + test all green · All API keys rotated

Complete Task Summary

Phase	Tasks	Data	Eng	Backend	ML Eng	Frontend	Full Stack	DevOps	Weeks
Phase 0 — Foundation	13	0	5	0	1	0	4	1	
Phase 1 — Data	11	6	3	0	0	1	0	2	
Phase 2 — Live Map	10	1	5	0	2	1	0	2	
Phase 3 — ML Models	9	0	1	5	1	1	0	3	
Phase 4 — Alerts	9	0	6	0	1	1	0	2	
Phase 5 — Rerouting	8	1	3	0	2	1	0	2	
Phase 6 — Copilot	4	0	2	0	1	1	0	1	
Phase 7 — Analytics	8	1	0	0	3	2	3	2	
TOTAL	72	9	25	5	11	8	7	15	

MINIMUM DEMO-ABLE BUILD (6 WEEKS)

Phase 0 (T-001→T-013) + Phase 1 (T-014→T-024) + Phase 2 (T-025→T-034) + Phase 4 partial (T-044→T-050)
= **~47 tasks** delivering a live map with real ships, DataCo shipments, manual disruption trigger, and financial alert. Already portfolio-worthy and demo-ready at week 6.

...